PakaonAI — Production-Readiness Assessment

Oct 12, 2026 · @Jeff Martinez

Verdict

PakaonAI is **not industry-grade yet**. It is a strong capstone and pilot system, but it should not feed real fish unattended.

This is based on the code on Jeff's computer, a full (non-shallow) clone of the repo, Vercel deployments and runtime errors, Supabase logs, and a lint, typecheck and unit-test run on the newest branch. Each finding below is marked **verified** (seen in code, logs or a test run) or **unverified** (a likely risk that still needs one check). No code was changed.

What is working

The foundations are solid for a student-built system, and every item here is verified.

- **Quality gates pass.** On the `Karl` branch, lint and typecheck are clean and all 109 unit tests (13 files) pass.
- **Production is up.** The live deployment is `main` at `d22e8cb`, and its last build is READY.
- **Ownership checks.** Every user action I read checks that the pond belongs to the signed-in user.
- **Careful device code.** Database writes in the device routes are transactional, events carry idempotency keys, and the firmware keeps an offline queue for feed events.
- **Safe boot.** The feeder relay defaults to OFF when the ESP32 starts.
- **Process.** CI blocks any API route that lacks an auth import, and commits follow Conventional Commits.

Branch state

Production runs `main`, but the newest work lives only on `Karl` and has never reached `main`. Checked in a full clone (not shallow).

Branch	Latest commit	State
`main`	`d22e8cb` (Aug 31)	Production. 55 commits ahead of `staging`, 12 ahead of `develop`
`staging`	`58b959b` (Jul 12)	Stale: 55 commits behind `main`, 0 ahead
`develop`	`0c86a87` (Aug 31)	Almost identical to `main`: 12 behind, 1 ahead (a merge commit only)
`Karl`	`b829247` (Oct 5)	3 commits ahead of `main`; deployed only as Vercel previews

The `Karl` commits carry the security-policy fix, the farm hierarchy, the biomass-based feed formula and 5 new test files. Production does not have any of them.

Verified gaps

Ten problems were confirmed in the code, logs or test run, listed roughly by impact.

#	Gap	What I saw
1	Unchecked feed amounts	`requestFeed(grams)` and the schedule settings accept any number, and the firmware turns grams straight into relay-on time with no cap. A typo of 100000 g would run the feeder about 7 hours.
2	Retries double-count feed	The firmware creates a new `event_id` on every retry. If the server saved an event but the device missed the reply, the retry is stored again, inflating grams and corrupting FCR.
3	Heartbeat flood	One heartbeat every 5 s writes about 34,500 rows per day per device, with no cleanup. A CRITICAL pellet notification is created on every heartbeat while the hopper is at 5% or less (about 17,000 a day).
4	No alert when a device dies	There are no scheduled jobs; the "disconnected" event is written only when the device comes back.

#	Gap	What I saw
5	Device routes untested	`/api/ingest`, `/api/feed-command` and `/api/schedule-sync` have no tests.
6	Roles not enforced	The UI calls VIEWER "read-only", but feed and schedule actions never check the role.
7	One shared database	Production, localhost dev and the Playwright tests all use the same Supabase project. HANDOFF.md records a past `prisma migrate reset` that wiped production data, and the build script runs `prisma migrate deploy` on every build, including CI.
8	CI and deploy gaps	CI runs only on pull requests to `main`, though the workflow says pull requests go to `develop`. The deploy workflow reads `secrets` in a job-level `if:`, so it looks broken; Vercel's Git integration is what actually deploys.
9	Thin monitoring	No error tracker; only `console` logging and a health check. Vercel's Hobby plan keeps 1 hour of runtime logs.
10	Production login errors	Vercel shows 5 `/login` errors in 7 days (latest Oct 12, 07:21 UTC): "tenant/user not found". The Supabase project itself is alive; the cause is unknown.

Likely but unverified

These are plausible from the code but not confirmed, so none is stated as fact. Each has a quick check.

Risk	Why it matters	How to check
Open sign-up with email-prefix identity	The app identifies a user by the part of the email before the `@` and ignores the domain. If public sign-up is on, anyone could register `farm-01@anything.com` and act as that farm.	In Supabase, confirm Auth "Allow new users to sign up" is off.
`User` table without row-level security	No migration enables it on `User`. Whether the public key can reach it depends on the Data API settings.	Run `select tablename, rowsecurity from pg_tables where schemaname='public'`.
Firmware TLS verification	The firmware sets no certificate, so it may not verify the server's identity, depending on the ESP32 core version.	Check the core version and test with a wrong certificate.
In-memory rate limiter	It lives in memory on serverless, so it probably does not hold across instances.	Load-test the login route.
Login lockout abuse	The lockout is per Farm ID, so someone could lock out another farm's account.	Try five bad logins on a test account.

Two things I did not count against the project. `registerEnergyDevice` has a missing-auth path, but nothing calls it, so it can simply be deleted. And the old security policies on `main` matched nothing (they fail closed); `Karl` already fixes them.

Recommended fix order

The first three fixes are small and protect the fish and the FCR data. The rest harden delivery.

1. Add limits on grams, feeding rate and feeds per day on the server, and a maximum relay time in the firmware.
2. Generate the `event_id` once when an event is queued, not on each retry.

3. Stop duplicate notifications, and thin out or expire heartbeat rows.
4. Confirm sign-up is off and run the `pg_tables` check. Then promote `Karl` to `develop` to `main`, give dev and tests their own Supabase project, and make CI run on `develop`.
5. Add tests for the three device routes and an error tracker.

Two decisions belong to Jeff, because they change production behavior: promoting `Karl` to production, and splitting the database.